

Teoria de Algoritmos

De La Cruz Flores José Rodolfo
Hernandez Cuella Carlos Perusi
Mendoza Roque Marcela

May 19, 2026

Heurísticos

Travelling Salesman Problem (TSP)

El Problema del Agente Viajero (TSP) es uno de los problemas de optimización combinatoria más estudiados en la investigación operativa y la ciencia de la computación. Dado un conjunto de ciudades y las distancias entre cada par de ellas, el TSP busca encontrar la ruta más corta que visite cada ciudad exactamente una vez y regrese al punto de partida.

Este problema pertenece a la clase de problemas NP, lo que significa que no se conoce un algoritmo que lo resuelva en tiempo polinomial para todas las instancias. Por esta razón, los enfoques prácticos recurren a métodos heurísticos y metaheurísticos que, sin garantizar optimalidad, producen soluciones de buena calidad en tiempos aceptables.

Planteamiento del Problema:

Un vendedor debe visitar un conjunto de n ciudades, partiendo de una ciudad inicial, visitando cada ciudad exactamente una vez, y finalmente regresando a la ciudad de origen. El objetivo es minimizar la distancia total recorrida o el costo asociado al recorrido.

Conocimientos Matemáticos:

- Sea $G = \{1, 2, \dots, n\}$ el conjunto de ciudades
- Una ruta se representa como una permutación $X = (x_0, x_1, \dots, x_{n-1})$ de G , donde x_k es la k -ésima ciudad visitada.
- El costo total de la ruta X es:

$$C(X) = \sum_{x=0}^{n-1} M[x_i x_{i+1}] + M[x_{n-1} x_0]$$

donde M es la matriz de costos tal que $M[i, j] = \text{costo}(i, j)$.

Modelo de Programación Entera:

Una formulación clásica en programación entera utiliza variables binarias $x_{ij} \in \{0, 1\}$ que indican si la arista (i, j) es utilizada en la ruta.

$$\text{Minimizar} \quad \sum_{i=1}^n \sum_{j=1}^n d_{ij} x_{ij} \quad (1)$$

$$\text{sujeto a:} \quad \sum_{j=1}^n x_{ij} = 1, \quad \forall i \in G \quad (2)$$

$$\sum_{i=1}^n x_{ij} = 1, \quad \forall j \in G \quad (3)$$

$$\sum_{i \in S} \sum_{j \notin S} x_{ij} \geq 2, \quad \forall S \subset G, S \neq \emptyset, S \neq G \quad (4)$$

$$x_{ij} \in \{0, 1\}, \quad \forall i, j \in G \quad (5)$$

Lo que nos indica (1) es minimizar la distancia total, teniendo restricciones (2) y (3) aseguran que cada ciudad tenga exactamente una arista de salida y una de entrada (grado 2), la restricción (4) garantiza que la solución sea un único ciclo que incluya todas las ciudades.

Algoritmos Implementados:

Operador de Mutación: 2-OPT

Es conocido como **Movimiento 2-opt**, dado un circuito $X = [x_0, x_1, \dots, x_{n-1}]$ e índices $0 \leq i < j \leq n - 1$ con distancia al menos 2, se define:

$$X_{ij} = [x_0, \dots, x_i, x_j, x_{j-1}, \dots, x_{i+1}, x_{j+1}, \dots, x_{n-1}]$$

La ganancia (disminución de costo) al aplicar un movimiento 2-opt es:

$$G(X, i, j) = M[x_i, x_{i+1}] + M[x_j, x_{j+1}] - M[x_{i+1}, x_{j+1}] - M[x_i, x_j]$$

El algoritmo 2-OPT explora posibles intercambios de aristas en la ruta mediante un proceso iterativo. Para cada par de aristas no adyacentes existen dos opciones:

- Mantener las aristas originales.
- Intercambiarlas invirtiendo el segmento intermedio.

Ejemplo:

Ruta original: A - B - C - D - E

Cambio: A - D - C - B - E

Con este algoritmo 2-opt converge rapidamente a un óptimo local, garantizando fácil de calcular en $O(1)$, su función principal es mejorar cualquier solución inicial, pero también esta la complejidad $O(n^2)$ por iteración, hasta $O(n^3)$ en el peor caso, puede quedar atrapado en óptimos locales, no garantiza encontrar la solución global óptima y para $n > 1000$, puede ser muy lento.

Cruce Parcialmente Emparejado

Dadas dos permutaciones α y β (padres) y dos puntos de cruce $j \leq k$, se construyen los hijos γ y δ mediante:

1. Copiar el segmento $[\alpha_j, \dots, \alpha_k]$ a γ y $[\beta_j, \dots, \beta_k]$ a δ .
2. Rellenar las posiciones restantes usando el mapeo de los padres.
3. Intercambiar los símbolos en conflicto.

En este algoritmo busca preservar la validez de las permutaciones (sin elementos repetidos), manteniendo el orden relativo de los genes, es biyectivo (dos padres producen dos hijos) volviéndose fácil de implementar. Por otro lado No se puede preservar buenas subrutinas, la complejidad $O(n^2)$ en la implementación ingenua la sensible a la elección de puntos de cruce y según Kreher & Stinson, PMX es inferior a MGKREC para TSP.

Algoritmo Genético Completo para TSP:

El algoritmo genético para el TSP busca rutas de costo mínimo representadas como permutaciones de ciudades. Parte de una población inicial aleatoria que se evalúa mediante una función de costo; las mejores soluciones se seleccionan para reproducirse. A través de operadores como el cruce (PMX) y la mutación, se generan nuevas rutas, manteniendo diversidad y evitando óptimos locales. Este proceso se repite por varias generaciones hasta cumplir un criterio de paro. Aunque no garantiza la solución óptima, produce soluciones de alta calidad en tiempos razonables para problemas grandes.

Resultados

El algoritmo genético para el TSP busca rutas de costo mínimo representadas como permutaciones, partiendo de una población inicial aleatoria que evoluciona mediante selección, cruce y mutación. Aunque no garantiza el óptimo, obtiene soluciones de alta calidad en tiempos razonables. Los operadores PMREC y MGKREC son métodos de recombinación, donde PMREC mezcla información de los padres, mientras que MGKREC incorpora un enfoque greedy, logrando mejores resultados.

Instancia	Algoritmo	Costo Promedio	Óptimos
M50a	PMREC	191.7	0
M50a	MGKREC	189.2	3
M50b	PMREC	175.4	0
M50b	MGKREC	161.6	3
M50c	PMREC	167.4	0
M50c	MGKREC	160.3	6

Contexto del problema

The knapsack problem

The knapsack problem también conocido como **El problema de la mochila**, El problema de la mochila es un problema clásico de optimización combinatoria. Consiste en seleccionar un conjunto de objetos con pesos y utilidades asociadas, de manera que el peso total no exceda la capacidad máxima de una mochila y la utilidad total sea máxima.

Este problema tiene aplicaciones en logística, asignación de recursos, finanzas e inteligencia artificial. Debido a que el número de combinaciones posibles crece exponencialmente conforme aumenta el número de objetos, existen distintos enfoques para resolverlo.

Se considera la versión 0/1 del problema, donde cada objeto puede ser seleccionado completamente o no seleccionado, convirtiendo un si o un no ya que en este problema no se usara fracciones.

En este proyecto se implementaron tres algoritmos para el problema de la mochila.

- Algoritmo Greedy
- Backtracking
- Recocido Simulado

Planteamiento Matemático

Sea un conjunto de n objetos, donde cada objeto tiene valor p_i y peso w_i y una mochila con capacidad máxima M . Por otro lado $x_i \in \{0, 1\}$ representa si el objeto es seleccionado o no.

El objetivo es maximiza el valor de los objetos es decir

$$\max \sum_{i=1}^n p_i x_i$$

sujeto a:

$$\sum_{i=1}^n w_i x_i \leq M$$

Donde se busca que el peso cada objeto seleccionado sea menor que la capacidad máxima

Algoritmos Implementados

Greedy

El algoritmo greedy ordena los objetos según la relación del valor y del peso, selecciona aquellos con mejor proporción mientras exista espacio disponible.

Su complejidad aproximada es $O(n \log n)$ debido al proceso de ordenamiento. Aunque es un algoritmo rápido, no siempre garantiza encontrar la solución óptima. En otras palabras se calcula de la siguiente manera Calcular p_i/w_i se procedera a Ordenar objetos y finalmente Ir agregando mientras haya capacidad

Backtracking

El algoritmo de backtracking explora todas las posibles combinaciones de objetos mediante recursión. El algoritmo construye un árbol de decisiones donde en cada nivel se decide incluir o no un objeto. Para cada objeto existen dos opciones:

- incluirlo
- no incluirlo

Este método garantiza encontrar la solución óptima, aunque su complejidad es exponencial $O(2^n)$

Recocido Simulado

El recocido simulado es una metaheurística inspirada en el enfriamiento de metales. El algoritmo genera soluciones vecinas y, en algunos casos, acepta soluciones peores para evitar quedar atrapado en óptimos locales.

La probabilidad de aceptar una solución peor está dada por $e^{\Delta/T}$ donde Δ representa el cambio en la utilidad y T la temperatura.

actual.solución = vector binario

vecino = cambiar un $0 \leftrightarrow 1$

Resultados

Se realizaron pruebas con una instancia pequeña del problema de la mochila.

Algoritmo	Valor obtenido	Tipo
Greedy	160	Heurístico
Backtracking	220	Exacto
Recocido Simulado	220	Metaheurístico

Los resultados muestran que el algoritmo greedy es rápido pero no siempre óptimo, mientras que backtracking encuentra la mejor solución con mayor costo computacional. El recocido simulado logró obtener una solución óptima aproximada con menor complejidad.

Conclusiones

En este proyecto se analizaron el Problema del Agente Viajero (TSP) y el Problema de la Mochila 0/1, comparando métodos exactos, heurísticos y metaheurísticos. En la mochila, el algoritmo greedy es rápido pero no siempre óptimo, mientras que backtracking garantiza optimalidad con alto costo computacional, y el recocido simulado ofrece un buen equilibrio. Para el TSP, se combinó generación lexicográfica con 2-OPT, logrando buenas soluciones en tiempos razonables. En general, los métodos exactos son útiles para instancias pequeñas, mientras que heurísticas y metaheurísticas son más eficientes para problemas grandes.

Bibliografía

1. Cormen, T., Leiserson, C., Rivest, R. y Stein, C. *Introduction to Algorithms*. MIT Press, 2009.
2. Kleinberg, J. y Tardos, É. *Algorithm Design*. Pearson, 2006.
3. Martello, S. y Toth, P. *Knapsack Problems: Algorithms and Computer Implementations*. Wiley, 1990.
4. Kirkpatrick, S., Gelatt, C. y Vecchi, M. *Optimization by Simulated Annealing*. Science, 1983.
5. Croes, G. A. (1958). A method for solving traveling-salesman problems. *Operations Research*, 6(6), 791-812.
6. Lawler, E. L., Lenstra, J. K., Rinnooy Kan, A. H. G., & Shmoys, D. B. (1985). *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*. Wiley.
7. Applegate, D. L., Bixby, R. E., Chvátal, V., & Cook, W. J. (2006). *The Traveling Salesman Problem: A Computational Study*. Princeton University Press.